MyPowerPlant · Testing & Verification

# Epic 4 Test Report

Security testing of the discharge-timing alert and calendar-export feature: injection testing against the compiled .ics output, one confirmed vulnerability, fix, and permanent regression coverage.

| 11 | 11 | 2 | 2 |
|---|---|---|---|
| TESTS RUN | PASSING | DEFECTS FOUND | DEFECTS FIXED |

| | | |
|---|---|---|
| PROJECT<br>**MyPowerPlant** | BRANCH<br>**fix/r22-date-field-validation** | DATE<br>**17 September 2026** |
| SCOPE<br>**R22 — .ics / alert field sanitisation** | STACK<br>**React 19 / TS (frontend only)** | STATUS<br>**Suite green, both defects fixed** |

## CONTENTS


1. Executive summary — what was tested, what it found
2. Testing approach — live attack simulation, real-parser verification
3. Automated test results — 11 tests, one file
4. Defects identified — one confirmed injection, one conformance gap
5. Verification by protocol-level parsing — independent check via a real RFC 5545 parser
6. Regression proof — tests confirmed to fail pre-fix
7. Continuous integration — current state: not wired up
8. Testing not yet executed — scope boundary, stated explicitly
9. Coverage gaps and recommendations — what to do next
10. Appendix A — Test inventory
11. Appendix B — Reproducing these results

## 1. Executive summary

This report covers security testing of Epic 4 (discharge timing alerts: guidance alerts plus an "add to calendar" export), scoped to Security Plan R22 — calendar-file and alert-content injection. Epic 4 lives on `feat/US4.1–4.4/reminder`, not yet merged to `main` or `sec`. Before this work the branch had no test runner configured at all.

Eight adversarial payloads were run through the compiled .ics output — the same class of attack R22 names explicitly — using the feature's real, unmodified source code, not a reimplementation. Seven were confirmed safe. One was a genuine, structural calendar-command injection: a crafted value in the `date` field, sourced from the Open-Meteo forecast API, produced a file that a real RFC 5545 parser could not read back because it contained an actual extra `VALARM` component.

The vulnerable field is fixed, the fix is independently verified against both the exploit and legitimate input, and 11 tests now guard the finding permanently.

> **Scope statement.** This report covers the R22 security surface only — the .ics/alert field sanitisation named explicitly in the plan for this epic. It is not a functional or usability test pass of the Get Reminded tab, and it does not include the Iteration 1 regression set (headers, CORS, rate limiting) because Epic 4 is not merged anywhere with a running instance to test against. Section 8 states this boundary again.

## 2. Testing approach

Two techniques, matching how the Epic 3 report argued its own findings needed independent calculation rather than inspection to surface:

- **Live attack simulation against the real source.** The actual `calendarExport.ts` exports (`buildIcsFile`) were bundled with esbuild and executed directly — not copied, not reimplemented — and fed crafted payloads: CRLF and bare-LF calendar-command injection, semicolon/comma field breaks, backslash double-escaping, control characters, an oversized value to stress line-folding, and a corrupted date field.
- **Authoritative parsing, not string matching.** An initial pass used a hand-written detector that searched the raw output for substrings like `BEGIN:VALARM`. That detector produced two false positives — it could not tell "appears as escaped text inside a folded field" from "appears as real calendar syntax". Re-verifying every case through `ical.js`, a real RFC 5545 parser, resolved this: it is the authoritative check reported in Section 5, and the reason the false positives did not make it into this report's findings.

## 3. Automated test results

| Suite | Layer | Tests | Result |
|---|---|---|---|
| `calendarExport.test.ts` | Frontend · new | 11 | **Pass** |
| **Total** | | **11** | **All passing** |

No pre-existing frontend or backend tests exist on this branch to report against — see Section 7.

## 4. Defects identified

**R22-1 — Unescaped date field allows structural .ics injection** MAJOR FIXED

`entry.date` — sourced from the Open-Meteo forecast response (`solarForecast.ts`: `date: data.daily.time[i]`) — was interpolated directly into `UID`, `DTSTART`, and `DTEND` with no escaping and no shape validation. Only `SUMMARY` / `DESCRIPTION` passed through `escapeIcsText()`.

A crafted date containing real CRLF sequences produced a file with an actual extra `VALARM` component — confirmed structural, not theoretical, because `ical.js` failed to parse the result back at all (`invalid line (no token ";" or ":")`).

**Impact.** Not reachable through today's UI in the classic "attacker types text" sense — but `date` is populated from a third-party API response, which is exactly the kind of trust boundary R20 and R22 both treat as untrusted by design. A malformed or compromised forecast response is a real, external path to this field.

**Fix.** `assertValidIsoDate()` rejects anything that is not a real `YYYY-MM-DD` date before `entry.date` is used anywhere — a bad value now throws instead of silently producing an exploitable file.

**R22-2 — Control characters not stripped from escaped text** MINOR FIXED

`escapeIcsText()` escaped backslash, semicolon, comma, and CRLF, but not other C0 control characters (null byte, vertical tab, etc.), which RFC 5545 forbids in TEXT values. `ical.js` parsed these without complaint either way, so this is a conformance gap, not a structural injection risk on its own — found during the same pass, fixed alongside R22-1 for consistency with the equivalent control-character stripping already applied to profile data in `profileStorage.ts`.

## 5. Verification by protocol-level parsing

Where the Epic 3 report independently hand-calculated KPI values rather than trusting the code under test, this report independently parsed the generated files with a real calendar library rather than trusting a hand-written detector. Every case was written to its own `.ics` file and read back with `ical.js`:

| Case | VEVENT | VALARM | Verdict |
|---|---|---|---|
| Legitimate entry | 1 | 0 | **Safe** |
| CRLF injection via `reason` | 1 | 0 | **Safe** |
| Bare LF injection via `reason` | 1 | 0 | **Safe** |
| Semicolon/comma break via `reason` | 1 | 0 | **Safe** |
| Backslash double-escape via `reason` | 1 | 0 | **Safe** |
| Control characters via `reason` | 1 | 0 | **Safe** |
| Oversized `reason` (line-fold stress) | 1 | 0 | **Safe** |
| CRLF injection via `date` (pre-fix) | — | — | **Parser rejected the file — confirmed injection** |

## 6. Regression proof

To confirm the new tests actually exercise the fix rather than passing vacuously, the pre-fix version of `calendarExport.ts` (the prior commit) was temporarily restored with the new test file kept in place. The date-validation tests failed against it, as expected; re-restoring the fixed file returned the suite to 11/11. This is the same discipline the Epic 3 report's hand-calculation section argues for: a test that cannot fail is not evidence.

## 7. Continuous integration

This branch has no CI workflow at all — unlike `sec`, which runs `security.yml` (secret scanning, dependency audit, lint, and both test suites) on every push. `vitest` was added to this branch specifically to run these 11 tests locally; there is no automated gate re-running them yet. This is listed as a recommendation in Section 9, not treated as already covered.

## 8. Testing not yet executed

- **Iteration 1 regression set** (security headers, CORS, HTTP methods, rate limiting) — not run against Epic 4, because it is not merged anywhere with a live instance to test against.
- **Manual and usability testing** of the Get Reminded tab itself (dismiss controls, the calendar download buttons, the forecast-driven day recommendations) — out of scope for this security-focused pass.
- **R17–R19** (microphone consent, voice data handling) belong to Epic 5, not Epic 4, and are not covered here.

One adjacent check was performed and is worth recording precisely: the alert text rendered in the Get Reminded tab (`RemindersView.tsx`) uses plain JSX interpolation (`{entry.reason}`, `{entry.dayLabel}`) with no `dangerouslySetInnerHTML` anywhere in the component — React escapes this by default, so there is no cross-site-scripting risk in the on-screen alert display itself. This was a targeted check, not a full review of the component.

## 9. Coverage gaps and recommendations

1. **Wire `calendarExport.test.ts` into CI** once this branch (or its fix) merges — either into `feat/US4.1-4.4/reminder`'s own pipeline or into `sec`'s existing `security.yml`.
2. **Run the Iteration 1 regression set** against Epic 4 once it has a live instance, matching what was already done for Epic 3.
3. **Resolve the 3 pre-existing lint errors** this branch still carries (2 `react-hooks/set-state-in-effect`, 1 `no-control-regex`) — already fixed on `sec`, not yet on this lineage.
4. **No further .ics defects expected** from this class of attack — both known fields that build the file (text content and date) are now validated or escaped.

## Appendix A — Test inventory

| Test | Checks |
|---|---|
| builds a well-formed single event for a normal reason | Baseline: exactly 1 VEVENT, 0 VALARM |
| escapes a CRLF calendar-command injection attempt to literal text | R22-1 class attack, safe path |
| escapes a bare LF injection attempt (no CR) to literal text | R22-1 class attack, safe path |
| escapes semicolons and commas rather than letting them break the field | RFC 5545 field-delimiter escaping |
| escapes backslashes before other escaping | Escape-of-escape ordering correctness |
| strips control characters rather than passing them through | R22-2 fix |
| folds an oversized reason without exceeding 75 octets | RFC 5545 line-folding correctness |
| builds normally for a real ISO date | R22-1 fix: no false rejection |
| rejects a CRLF-injection payload smuggled through the date field | R22-1 fix: the confirmed vulnerability |
| rejects a non-ISO date format | R22-1 fix: shape validation |
| rejects a shape-valid but impossible calendar date (2026-13-45) | R22-1 fix: real-date validation |

## Appendix B — Reproducing these results

```
git fetch origin fix/r22-date-field-validation
git checkout fix/r22-date-field-validation
cd frontend
npm install
npm test                    # calendarExport.test.ts — 11 tests
npm run build               # confirms tsc + vite build are clean
```